

CS202-Lab3 – Report

Hunter Adomitis(hadom001)

Iftikhar Kalanther (ikala005)

Joseph D'Angelo(jdan003)

In this project, we created a system call `clone()`, and three user-level library routines, `thread_create()`, `lock_acquire()`, and `lock_release()`, in xv6-riscv. After the changes, the kernel-level thread support and thread management were tested using the multi-threaded test program `test3_lab.c` as given in the lab3.pdf document. The entire XV6 source code with our modifications is available in the GitHub repository at <https://github.com/Iftikhar31/cs202-lab3-xv6-riscv.git>. The video demo for this lab 3 assignment is available at <https://drive.google.com/file/d/1fx7iAb5A5FiKFJDCUO2ril9HnrzD1ZUP/view?usp=sharing>.

1. Modified files:

The part1 – `clone()` system call changes are in the kernel folder, and the part2 – user-level thread library changes are in the user folder.

The following files were modified to create and make `clone()` system call and user-level library routines.

kernel folder:

1. `defs.h`
2. `proc.c`
3. `proc.h`
4. `syscall.c`
5. `syscall.h`
6. `sysproc.c`
7. `trampoline.S`
8. `trap.c`

user folder:

1. `lab3_test.c`
2. `thread.c`
3. `thread.h`
4. `user.h`
5. `usys.pl`

CS202-lab:

1. `Makefile`

2. Changes and screenshots:

- kernel/syscall.h:

syscall.h was modified to define new number 27 for the syscall – clone() - as shown in the screenshot below.

```
27      #define SYS_sched_tickets 26
28      #define SYS_clone 27
```

- kernel/syscall.c:

- Declared clone().

```
112      extern uint64 sys_clone(void);
```

- Updated the system call table (pointer array) *syscalls[] with clone system call.

```
142      [SYS_clone] sys_clone
```

- kernel/sysproc.c (sys_clone):

- Defined syscall function 'sys_clone'. This function receives the allocated stack from the user space and returns the 'tid' (thread id) to the user space once a thread is created. For the main thread (parent), the tid returned will be the child thread's tid. The tid returned will be zero for the newly created child's thread.

```
175      uint64 sys_clone(void)
176      {
177          uint64 addr;
178          argaddr(0,&addr);
179          return clone((void*)addr);
180      }
```

-

- kernel/proc.h:

- The proc struct definition in proc.h is updated to include 2 variables – threadid and numThreads. 'threadid' will hold the id of the thread, and 'numThreads' is used to assign new thread id, when a thread is created. For the first child thread, it will be 1 and will be incremented by 1 for subsequent thread creations.

```

94         int threadid;
95         int numThreads;

```

○

- kernel/proc.c:

allocproc():

- The allocproc() function is modified to initialize 'threadid' to zero.

```

130         p->threadid=0;

```

freeproc():

- The freeproc() function is modified to free the page table only if it is called by the parent thread (thread id == zero). Otherwise, only the thread's trapframe will be unmapped. The trapframe address is calculated based on the thread-id using 'TRAPFRAME - PGSIZE * (p->threadid)' where TRAPFRAME is the starting address of the main parent thread.

```

170         if(p->threadid==0){
171             if(p->pagetable)
172                 proc_freepagetable(p->pagetable, p->sz);
173         }
174         else{
175             uvmunmap(p->pagetable, TRAPFRAME - PGSIZE * (p->threadid), 1, 0);
176         }

```

exit():

- The exit() is modified to close the files only if it is called by the parent thread (thread id == 0).

```

367         if(p->threadid == 0){
368             // Close all open files.
369             for(int fd = 0; fd < NOFILE; fd++){
370                 if(p->ofile[fd]){
371                     struct file *f = p->ofile[fd];
372                     fclose(f);
373                     p->ofile[fd] = 0;
374                 }
375             }
376         }

```

allocproc_thread():

- A new function, allocproc_thread(), was created by copying allocproc(). This allocproc_thread() function, when called, will assign values to all fields for a particular thread. The code modified will assign 'threadid' by incrementing the 'numThreads'. It will map a trapframe page to the thread based on 'TRAPFRAME - PGSIZE * (p->threadid)'. To be safe, if 'mappages' fails for any reason, it will unmap it.

```
allocproc_thread()
```

```

{
    struct proc *t;
    int threadn;
    struct proc* p = myproc();
    acquire(&p->lock);
    p->numThreads++;
    threadn=p->numThreads;
    release(&p->lock);

```

found:

```

    t->pid = allocpid();
    t->threadid = threadn;

    if(mappages(p->pagetable, TRAPFRAME - PGSIZE * t->threadid, PGSIZE,
        (uint64)(t->trapframe), PTE_R | PTE_W) < 0){
        uvmunmap(p->pagetable, TRAPFRAME - PGSIZE * (t->threadid), 1, 0);
    }
    return 0;
}

```

clone():

- A new function, clone(), is created by copying fork(). This function is called to create a new thread. When called, it will call allocproc_thread() to assign appropriate values for the newly created thread. The copy page table was removed as the threads will share the page table. The trapframe's stack pointer (sp) is assigned a value by adding the stack value passed by the user-level routine with page PGSIZE (4096) as the stack is growing downwards.

```
int clone(void * a){  
  
    int i, tid;  
  
    struct proc *np;  
    struct proc *p = myproc();  
  
  
    // Allocate process.  
  
    if((np = allocproc_thread()) == 0){  
        return -1;  
    }  
  
    np->pagetable=p->pagetable;  
  
    np->trapframe->sp=(uint64)a+PGSIZE;
```

- kernel/defs.h:

- The clone() function is declared in the proc.c section.

- kernel/trampoline.S:

- The trampoline.S was replaced with the one provided in the lab3.pdf document.

- kernel/trap.c:

- The trap.c was modified to tell the kernel the trapframe locations of the child thread. The parent thread (thread id == 0) will use the default trapframe address.

```
130     if(p->threadid==0){
131         ((void (*)(uint64,uint64))trampoline_userret)(TRAPFRAME, satp);
132     }
133     else{
134         ((void (*)(uint64,uint64))trampoline_userret)(TRAPFRAME - PGSIZE * p->threadid, satp);
135     }
136 }
```

The following changes are done in the user folder. The user-level thread library routines are implemented in thread.c and thread.h.

- user/thread.h:

The library routines thread_create, lock_init, lock_acquire, and lock_release were declared in thread.h. The lock_t struct was also defined.

```
1     struct lock_t {
2         uint locked; // 1 if the lock is held, 0 otherwise
3     };
4
5     typedef struct lock_t lock_t;
6
7     int thread_create(void *(start_routine)(void *), void *arg);
8     void lock_init(struct lock_t *lock);
9     void lock_acquire(struct lock_t *lock);
10    void lock_release(struct lock_t *lock);
```

- **user/thread.c:**

thread.c is created to implement thread library routines. It has a thread_create() routine, which allocates a stack of 4096 bytes and calls the clone() syscall by passing the allocated stack. If the clone() syscall returns a thread id of zero, then it is a child thread that is created, and hence the start_routine() entry function is called. The lock_init() initializes the lock to zero (locked). The lock_acquire routine implements '__sync_lock_test_and_set', which tests and assigns 1 to locked if the locked value is 0. If the previous locked value was 1, then it will spin until it becomes zero. Once zero, it will acquire the lock by setting the locked value to 1. The lock_release function implements '__sync_lock_release', which will release the lock by setting the locked value to 0.


```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4  #include "kernel/riscv.h"
5  #include "user/thread.h"
6
7  int thread_create(void *(start_routine)(void *), void *arg) {
8      // Allocate a stack of PGSIZE bytes for the thread
9      void *stack = malloc(PGSIZE);
10     if (stack == 0) {
11         return -1;
12     }
13
14     // Call clone with the allocated stack
15     int tid = clone(stack);
16     if (tid < 0) {
17         free(stack);
18         return -1;
19     }
20
21     // For the child thread, execute the start_routine with the argument arg
22     if (tid == 0) {
23         start_routine(arg);
24         exit(0);
25     }
26
27     // For the parent thread, return 0 on success
28     return 0;
29 }
30
31 void lock_init(struct lock_t *lock) {
32     lock->locked = 0; // Unlocked
33 }
34
35 void lock_acquire(struct lock_t *lock) {
36     // __sync_lock_test_and_set assigns &lock->locked to 1
37     // and returns the previous value. If the previous value was 1,
38     // the lock is already held, so we spin until it becomes 0.
39     while (__sync_lock_test_and_set(&lock->locked, 1) == 1) {
40         // weeeeeeee
41     }
42 }
43
44 void lock_release(struct lock_t *lock) {
45     // set lock->locked to 0 atomically
46     __sync_lock_release(&lock->locked);
47 }

```

- **user/usys.pl:**

- The user space syscall interface is updated by creating an entry for syscall 'clone()' in usys.pl.

```
44      entry("clone");
```

- **user/user.h:**

- User system call (clone()) was updated in the user-space system call interface user.h.

```
28      int clone(void *);    // lab3 clone
```

- **user/lab3 test.c:**

- This file provided in the lab3 instruction contains the code to test the user-level thread management and kernel-level thread support.

- **Makefile:**

- Makefile is updated by appending "\$U/_lab3_test\" to UPROGS.

```
140      $U/_lab3_test\
```

- This file is also updated to change the number of CPUs to 3. The ULIB value was changed to include thread.o so that the thread.c is compiled as a library and can be used by other user-level programs by including user/thread.h.

```

162         ifndef CPUS
163         CPUS := 3
164     endif

91     ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/thread.o

```

Testing:

The user-level thread management and kernel-level support were tested using lab3_test.c. The CPUs used were 3.

Frisbee program result (10 rounds and 3 threads):

xv6 kernel is booting

hart 1 starting

hart 2 starting

init: starting sh

\$ lab3_test 10 3

Round 1: thread 0 is passing the token to thread 1

Round 2: thread 1 is passing the token to thread 2

Round 3: thread 2 is passing the token to thread 0

Round 4: thread 0 is passing the token to thread 1

Round 5: thread 1 is passing the token to thread 2

Round 6: thread 2 is passing the token to thread 0

Round 7: thread 0 is passing the token to thread 1

Round 8: thread 1 is passing the token to thread 2

Round 9: thread 2 is passing the token to thread 0

Round 10: thread 0 is passing the token to thread 1

Frisbee simulation has finished, 10 rounds played in total

\$ █

Frisbee program result (21 rounds and 20 threads):

```
$ lab3_test 21 20
```

```
Round 1: thread 0 is passing the token to thread 1
Round 2: thread 1 is passing the token to thread 2
Round 3: thread 2 is passing the token to thread 3
Round 4: thread 3 is passing the token to thread 4
Round 5: thread 4 is passing the token to thread 5
Round 6: thread 5 is passing the token to thread 6
Round 7: thread 6 is passing the token to thread 7
Round 8: thread 7 is passing the token to thread 8
Round 9: thread 8 is passing the token to thread 9
Round 10: thread 9 is passing the token to thread 10
Round 11: thread 10 is passing the token to thread 11
Round 12: thread 11 is passing the token to thread 12
Round 13: thread 12 is passing the token to thread 13
Round 14: thread 13 is passing the token to thread 14
Round 15: thread 14 is passing the token to thread 15
Round 16: thread 15 is passing the token to thread 16
Round 17: thread 16 is passing the token to thread 17
Round 18: thread 17 is passing the token to thread 18
Round 19: thread 18 is passing the token to thread 19
Round 20: thread 19 is passing the token to thread 0
Round 21: thread 0 is passing the token to thread 1
Frisbee simulation has finished, 21 rounds played in total
$ █
```

Contributions:

The team discussed the high-level approach and decided to do the following by everyone.

1. Hunter Adomitis: Part 1 – Clone() system call and kernel-level changes. Review Part 2 user-level thread library changes and report.
2. Iftikhar Kalanther: Write the report. Review Part 1 – Clone() syscall and Part 2 user-level thread library changes. Applied the Part 1 and Part 2 changes made by the team to a fresh copy of xv6-riscv and tested them.

3. Joseph D'Angelo: Part 2 user-level thread library routines and changes. Review Part 1 – Clone() syscall and report.

All 3 members ran the test program lab3_test.c and verified the frisbee results for various rounds and threads. The report was reviewed by each team member before submission.